

Software Engineering

Dr. Md. Nasim Adnan

- He had the classic look of a senior executive for a major software company—mid-40s, slightly graying at the temples, trim and athletic, with eyes that penetrated the listener as he spoke.
- But what he said shocked me. “Software is *dead*.”
- I blinked with surprise and then smiled. “You’re joking, right? The world is driven by software and your company has profited handsomely because of it. It isn’t dead! It’s alive and growing.”
- He shook his head emphatically. “No, it’s dead . . . at least as we once knew it.”
- I leaned forward. “Go on.”

- He spoke while tapping the table for emphasis. “The old-school view of software—you buy it, you own it, and it’s your job to manage it—that’s coming to an end.
- Today, with Web 2.0 and pervasive computing coming on strong, we’re going to be seeing a completely different generation of software. It’ll be delivered via the Internet and will look exactly like it’s residing on each user’s computing device . . . but it’ll reside on a far-away server.”
- Is software *really dead? If it was, you wouldn’t be reading this book!*

- Computer software continues to be the single most important technology on the world stage. And it's also a prime example of the law of unintended consequences.
- Fifty years ago no one could have predicted that software would become an indispensable technology for business, science, and engineering.
- When Dijkstra married Maria in 1957, he was required as a part of the marriage rites to state his profession. He stated that he was a Programmer, which was unacceptable to the authorities, there being no such profession then in The Netherlands.

- No one could predict that millions of computer programs would have to be corrected, adapted, and enhanced as time passed.
- The burden of performing these “maintenance” activities would absorb more people and more resources than all work applied to the creation of new software.
- As software’s importance has grown, the software community has continually attempted to develop technologies that will make it easier, faster, and less expensive to build and maintain high-quality computer programs.

The Nature of Software

- Today, software takes on a dual role. It is a product, and at the same time, the vehicle for delivering a product. As a product, it delivers the computing potential embodied by computer hardware. As a vehicle for delivering a product, software acts as an information transformer — it produces, manages, modifies, displays, or transmits information. This information can be as simple as a single bit or as complex as a multimedia presentation created from data collected from many independent sources

- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).
- Software delivers the most important product of our time — *information*. *It transforms* personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks (e.g., the Internet), and provides the means for acquiring information in all of its forms.

- Today, a huge software industry has become a dominant factor in the economies of the industrialized world. Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era.
- And yet, the questions that were asked of the lone programmer are the same questions that are asked when modern computer-based systems are built:

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?



- Margaret Hamilton in 1969, standing next to listings of the software she and her MIT team produced for the Apollo project.

Defining Software

- **Software is:**

Instructions (computer programs): When executed, they provide the desired features, functions, and performance.

Data structures: These allow the programs to properly organize and manipulate information.

Descriptive information: This is available in both hard copy and virtual forms, and it explains the operation and use of the programs.

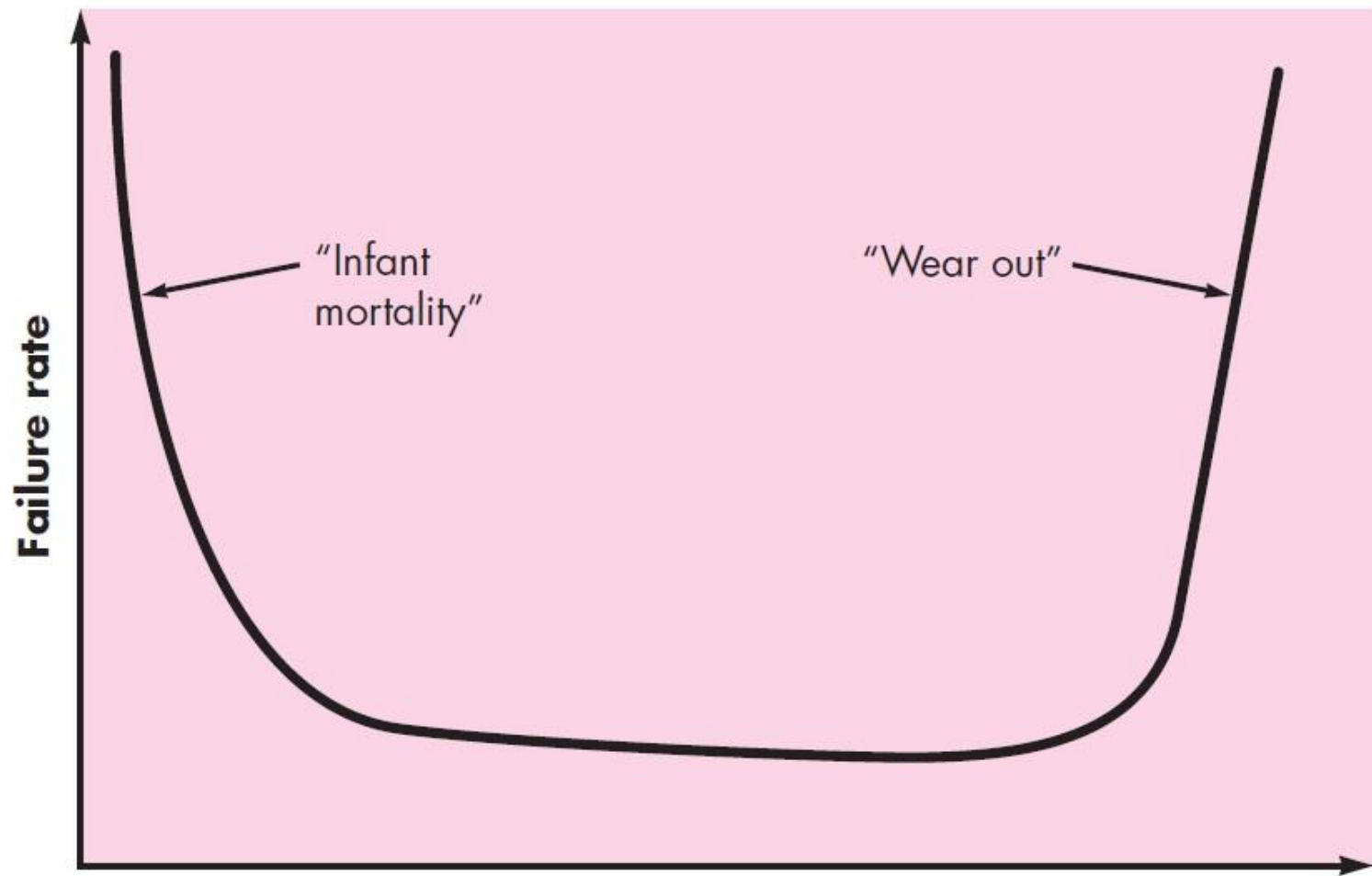
- Software has characteristics that are considerably different than those of hardware:
 - *Software is developed or engineered; it is not manufactured in the classical sense*
 - *Software doesn't "wear out."*
 - *Although the industry is moving toward component-based construction, most software is still custom built.*

The next Figure shows the failure rate of hardware over time, often called the “bathtub curve.”

At the beginning, hardware has a high failure rate due to design or manufacturing defects. After these defects are fixed, the failure rate drops to a steady, low level for some time.

As time goes on, the failure rate rises again because hardware parts are affected by dust, vibration, misuse, temperature extremes, and other environmental factors.

Simply put, over time, the hardware starts to wear out

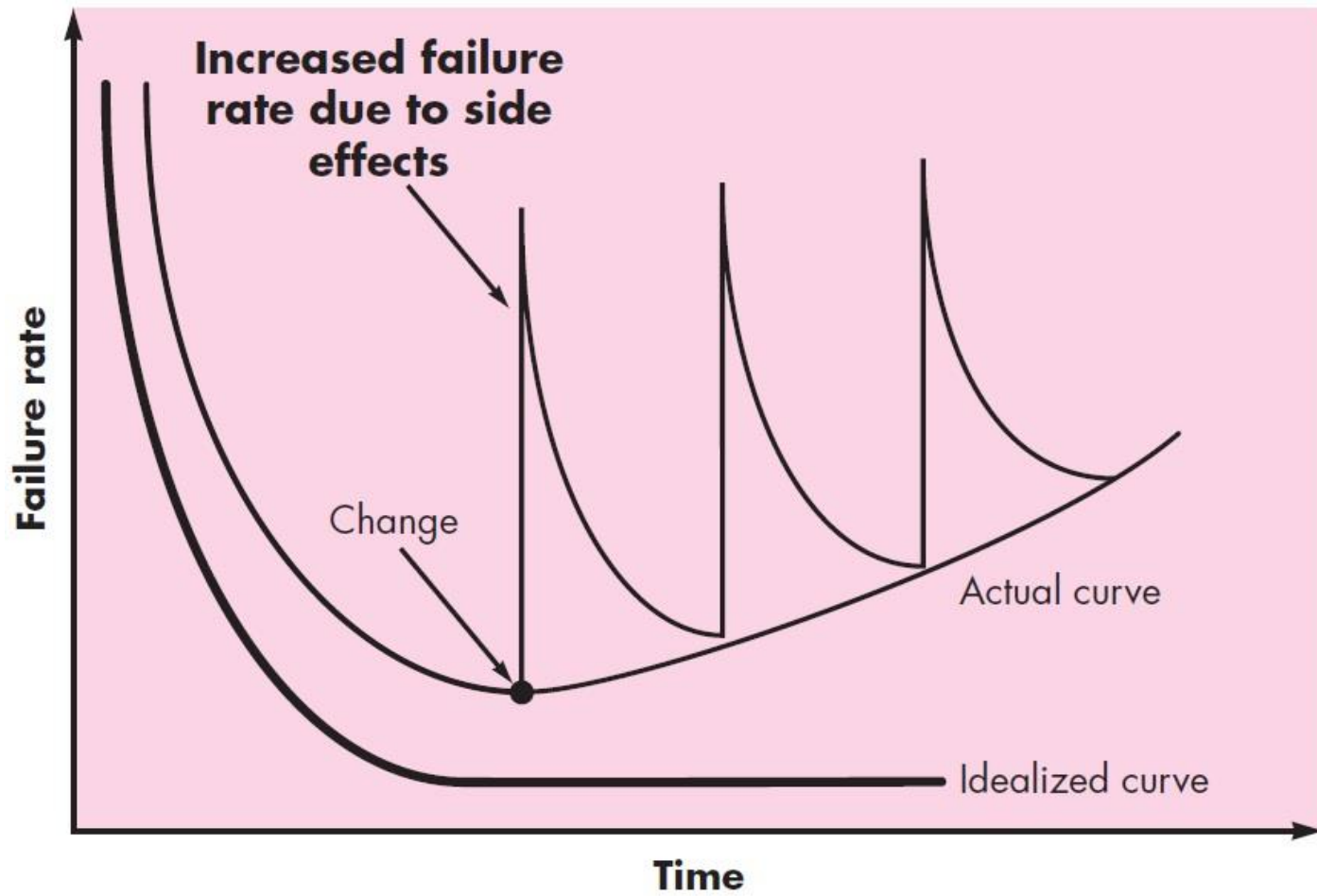


Software is not affected by the environmental problems that make hardware wear out. In theory, the failure rate curve for software should look like the “idealized curve” shown in the next Figure.

Undiscovered defects can cause high failure rates early in a program's life. Once these defects are fixed, the curve flattens as shown.

The idealized curve is a simplified version of actual software failure models.

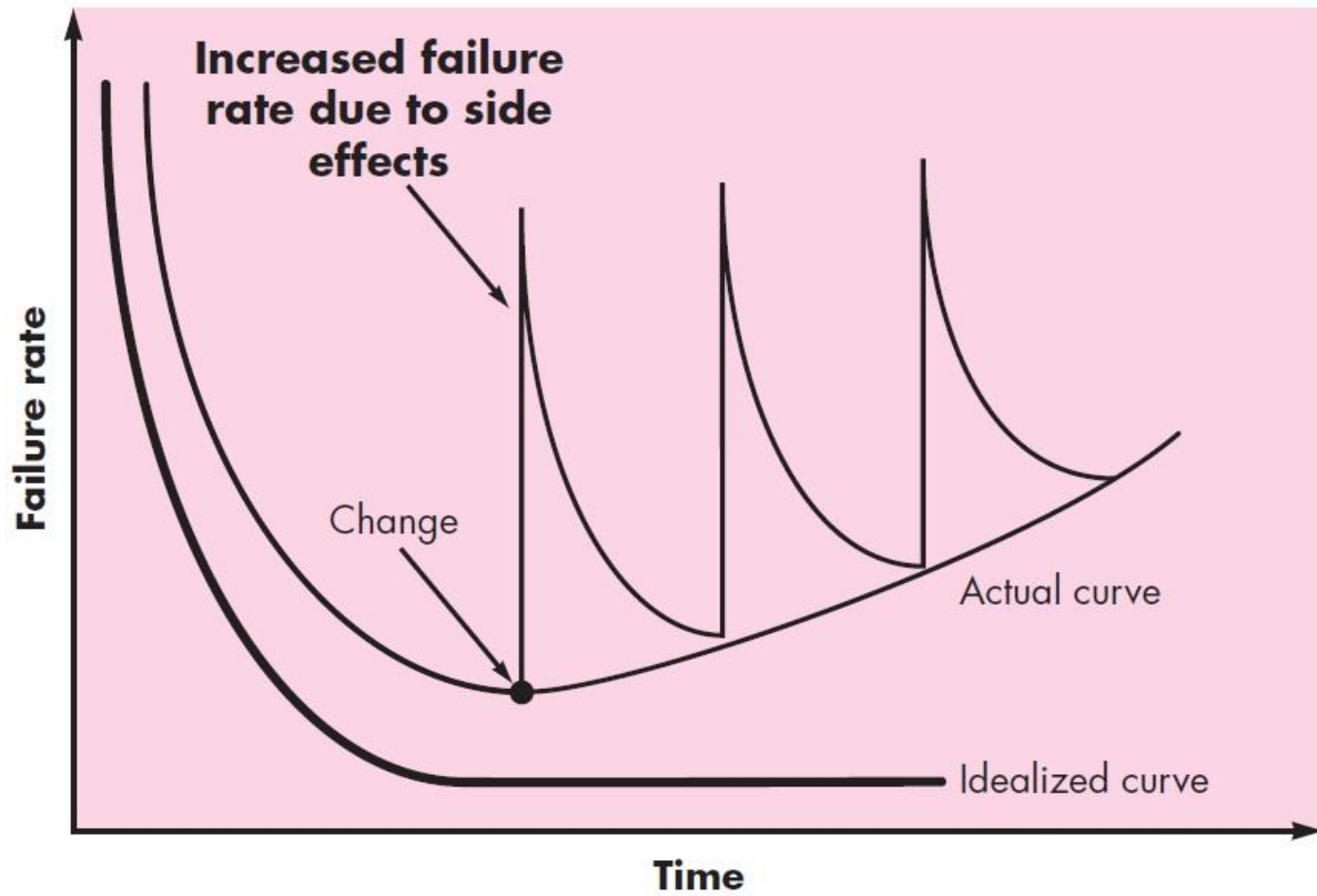
The key point is clear—software doesn't wear out, but it can still deteriorate!



Real- World Failure Curve (Actual Curve): As changes are made, it is likely that errors will be introduced, causing the failure rate curve to spike, as shown in the “actual curve.”

Before the curve can return to the original steady- state failure rate, another change may be requested, causing the curve to spike again.

Gradually, the minimum failure rate level starts to rise — this shows that the software is deteriorating due to changes.



Software Application Domains

- System Software
- Application Software
- Engineering/Scientific Software
- Embedded Software
- Product-line Software
- Web Applications
- Artificial Intelligence Software

"Software application domains" refer to the different areas or industries where software is applied to solve problems or provide services.

Software Engineering

- In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:
 - First, a concerted effort should be made to understand the problem before a software solution is developed.
 - Second, The information technology requirements of individuals, businesses, and governments are becoming more complex every year (for example, Income Tax systems). Now, large teams of people work together to create computer programs that were once developed by a single individual.

- People, businesses, and governments now depend more and more on software to make important decisions and run their daily work. If the software doesn't work properly, it can cause small problems — or even very serious failures. That's why **software needs to be high quality.**
- Also, when an application becomes more useful and valuable, more people will start using it, and it will likely be used for a long time. As this happens, people will want new features and updates. **So, the software should maintainable.**

Footnote

Footnote – Types of Information Systems:

DSS (Decision Support System)

Helps in long-term planning and making important decisions.

Example: Planning changes in staff or company structure.

MIS (Management Information System)

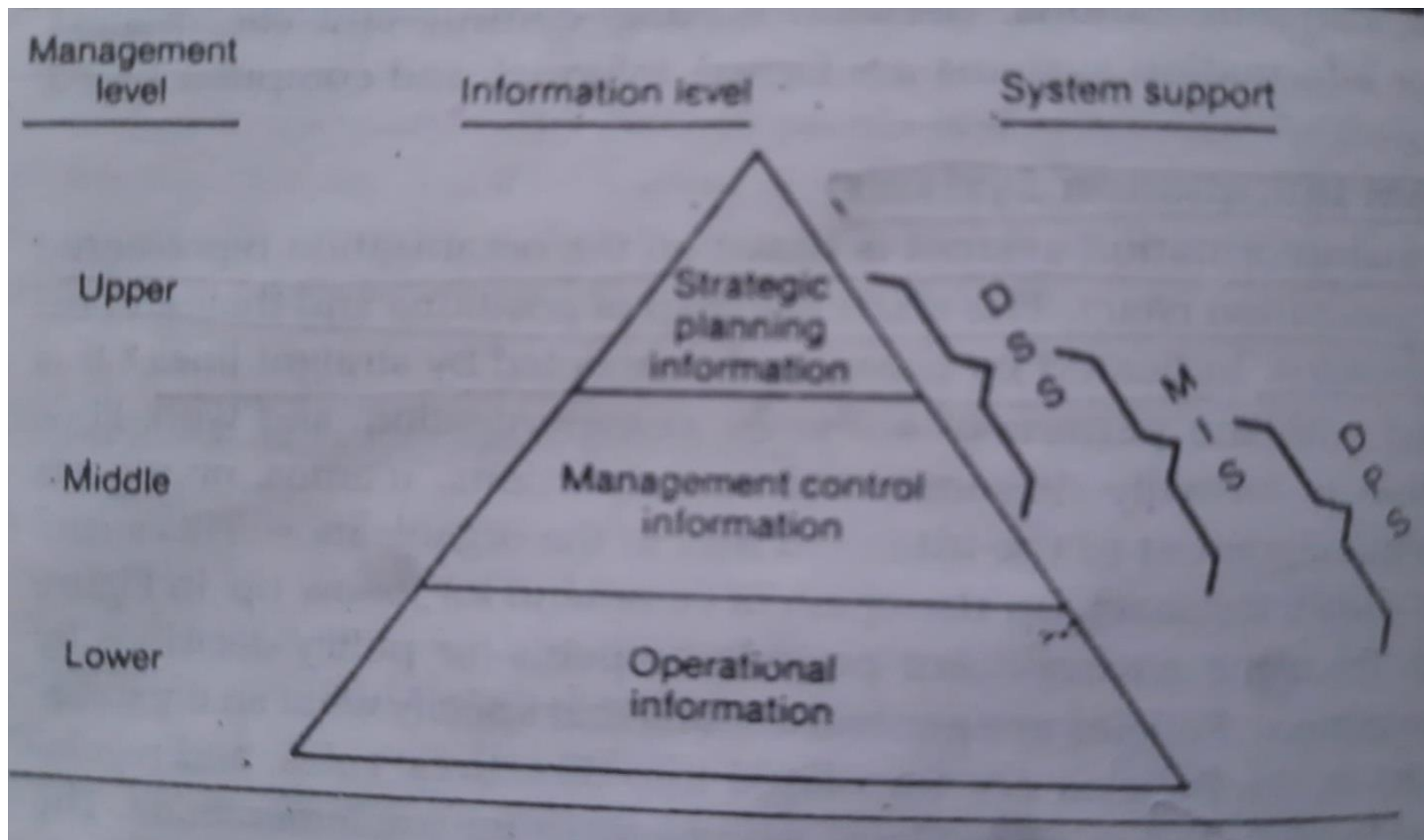
Most people are familiar with this.

Used for analyzing business performance, like monthly or yearly sales reports.

DPS (Data Processing System)

Enforce day to day rules and regulations of a business

Examples: Employee absence sheet, current stock of products



Focus Area: MIS

- A key element of MIS is the database. MIS cannot be implemented effectively without the help of a database
- A database is a nonredundant collection of interrelated data items that can be accessed through application programs and available to many users

Decision Making

❖ Decision Making – Elements:

Data/Information – Needed to understand the problem and options.

Problem/Goal – The issue to solve or the goal to reach.

Alternatives – Different choices or actions to consider.

Analysis – Comparing the options using facts and tools.

Choice – Picking the best option.

Implementation – Putting the chosen option into action.

Review – Checking if the decision worked well or needs change.

Decision Making

MIS (Management Information System)

Collects and organizes everyday business transactions.

Examples:

Production MIS – handles production data

Personnel MIS – handles employee data

The data is grouped and analyzed based on different dimensions, like time, department, or product.

DSS (Decision Support System)

In simple words, a DSS uses results from MIS systems along with external data (like market trends) to help managers make better decisions

All of these facts point to one clear idea:

No matter what type of software we build or where it is used, it should be carefully planned and developed—just like in engineering. This brings us to the main topic: software engineering.

Over time, many experts have given their own definitions of software engineering. But one of the most well-known and commonly used definitions was given by Fritz Bauer:

[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines

The Software Process

- *A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.*

1. An activity is a big step that works toward a general goal.

Example: Talking and planning with the people who need the software (stakeholders).

This happens in all types of projects, no matter the size or complexity.

2. An action includes several smaller tasks that help create an important part of the software.

Example: Making the overall design of the system (architectural design).

3. A task is a very specific job with a clear goal.

Example: Running a unit test to check a small part of the code.

This produces something you can see or use.

In software engineering, a process is not a strict set of rules that must always be followed the same way.

Instead, it's a flexible guide that helps the software team choose the right steps based on their project.

The main goal is always to deliver the software on time and with good enough quality to make both the client and the users happy.

Process Framework

- A generic process framework for software engineering encompasses five activities:
- **Communication.** Before any technical work, it is critically important to communicate and collaborate with the customer (and other stakeholders).
- The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

- **Planning.** Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a “map” that helps guide the team.
- The map—called a *software project plan*—defines the software engineering work by describing the **technical tasks** to be conducted, the **risks** that are likely, the **resources** that will be required, **the work products** to be produced, and a work **schedule**.
(Padma Bridge)

- **Modeling.** People in many fields — like architects, engineers, use models or sketches before starting their work. These models help them see the overall structure, understand how different parts fit together, and plan everything clearly. It gives them a big-picture view before they start building.

In the same way, software engineers create models to understand what the software should do and how it should be built. These models start simple and can be made more detailed as needed. This helps the team clearly understand the requirements and design a solution that works.

- **Construction.** This is the stage where the actual coding happens. Developers write the program manually or use tools to help generate code. Along with writing the code, they also test it to find and fix any errors, making sure everything works as expected .
- **Deployment.** Once the software (or part of it) is ready, it is delivered to the customer. The customer then tries it out and gives feedback based on their experience. This feedback helps the team know what's working well and what might need improvement.

- As discussed previously, a generic process framework for software engineering defines five framework activities—**communication, planning, modeling, construction, and deployment.**
- We now describe the principles that focus on software engineering practice as a whole.

The First Principle: The Reason It All Exists

- A software system exists for one reason: to provide value to its users. Every decision should be made with this idea in mind. Before writing a system requirements such as adding a feature, choosing hardware, or deciding on a process, always ask: “Does this really add value to the system ?” If the answer is “no ,” then don’ t include it. All other principles support this one.

The Second Principle: KISS (Keep It Simple, Stupid!)

- Software design is not a haphazard process. There are many factors to consider in any design effort. All design should be as simple as possible. This facilitates makes the system easier to understand and maintain.

The Third Principle: Maintain the Vision

- A clear vision is essential to the success of a software project. If there's no clear vision, the project can become confusing and disorganized, with different people pulling it in different directions. This usually creates a system that looks messy and has designs that don't fit well together.

Keeping a strong, consistent vision helps the whole system stay coherent, reliable, and easier to manage.

The Fourth Principle: What You Produce, Others Will Consume

- A software system is never built in isolation; others will use , maintain, document, or extend it. This means your requirements, designs, and code should always be clear and easy to understand. You should specify with the users in mind, design while thinking of the developers who will implement it, and write code that future maintainers can easily debug and extend. By making your work understandable, you make life easier for others — and that adds real value to the system.

The Fifth Principle: Be Open to the Future

- A system gains more value when it lasts longer. In today's fast-changing world, where requirements shift quickly and hardware becomes outdated in months, most software has a short life. But real industrial-strength systems must survive for years. To achieve this, they need to be designed from the start with adaptability in mind, so they can adjust to new requirements, technologies, and environments without breaking.

The Sixth Principle: Plan Ahead for Reuse

Reusing software saves both time and effort, but achieving effective reuse is one of the hardest challenges in system development. Object-oriented technologies promise benefits like code and design reuse, but these advantages do not come automatically. To truly gain from reuse, systems must be designed with foresight and proper planning, ensuring that components are flexible, adaptable, and easy to integrate into future projects.

The Seventh principle: Think!

- The final principle is often the most overlooked. Taking time to think clearly and completely before acting usually leads to better results. When you carefully consider a problem or decision first, you are far more likely to do it right the first time, saving time and effort in the long run.

